

Project Title: Student Performance Prediction Using Machine Learning

Name: Elamathi M

Roll No: 23AD016

Department: Artificial Intelligence and Data Science-III year

Date of Submission: 18-10-2025

2.Abstract

This project focuses on predicting student performance using machine learning techniques. The dataset contains various attributes related to students' academic, personal, and social factors. The primary goal is to predict the final scores of students based on these features, helping educators identify students who may need additional support. Data preprocessing steps, including handling missing values and encoding categorical variables, were performed to prepare the dataset for modeling. Exploratory data analysis revealed correlations between study habits, failures, and final scores, providing insights into key factors affecting performance. A neural network (MLP) model was trained on the processed dataset, and its performance was evaluated using metrics such as Mean Squared Error, Mean Absolute Error, and R^2 . The model successfully predicts student outcomes with reasonable accuracy, demonstrating that machine learning can be a useful tool in educational planning and intervention.

3.Introduction

Student performance is influenced by a wide range of factors, including demographics, study habits, family background, and school environment. Understanding these factors is essential for educators, schools, and policymakers to identify trends, improve teaching strategies, and provide targeted interventions for students who may be struggling. With the increasing availability of educational data, data-driven approaches can uncover hidden patterns and correlations that traditional methods might overlook. This project focuses on analyzing a real-world student dataset that includes personal, social, and academic features. Through exploratory data analysis (EDA) and visualization, we aim to gain a deeper understanding of how different factors relate to student performance. Furthermore, by applying a deep learning model (MLP), we attempt to predict final academic outcomes, providing actionable insights that can support better educational planning.

3.Objective

To implement a predictive model for student performance by exploring and analyzing a dataset, performing data preprocessing, visualizing important patterns, and building a neural network model (MLP). The project aims to generate insights that can help educators and policymakers identify at-risk students and improve academic strategies.

4.Dataset Description

The dataset used in this project contains information about students' personal, social, and academic attributes. It is collected from a publicly available educational source and aims to provide a comprehensive view of factors that may influence student performance.

- **Source:** The dataset is publicly available and widely used for student performance analysis.
- **Size:** The dataset consists of 396 students with 20 features.
- **Fields/Features:** The dataset includes a mix of categorical and numerical variables. Some key features are:
- **Personal attributes:** age, sex, address, family size, parents' education, etc.
- **Social attributes:** study time, extra-curricular activities, travel time to school, family support, etc.
- **Academic attributes:** previous grades, school absences, and other performance indicators.
- **Target variable:** The final academic outcome or score that we aim to predict.

Basic statistics:

- Numerical features are summarized using mean, median, and standard deviation.
- Categorical features are summarized by value counts to show the distribution of categories.

5.Exploratory Data Analysis (EDA) and Preprocessing

Methods Used:

Data Cleaning: Checked for missing values in all columns and handled them by imputing with mean for numerical features or mode for categorical features. Removed duplicates and irrelevant columns that do not contribute to the target variable.

Data Transformation: Categorical variables (e.g., gender, address type, parental education) were converted into numerical format using encoding techniques like Label Encoding or One-Hot Encoding. Numerical features were scaled using standardization or normalization to bring all variables onto a comparable scale.

Visualization for EDA:

1. **Histograms** and **box plots** were used to understand distributions and identify outliers.
2. **Bar plots** were used to analyze categorical feature distributions.
3. **Correlation heatmaps** were plotted to identify relationships between numerical variables and the target variable.
4. **Pair plots** and **scatter plots** helped visualize how features interact with each other and the target variable.
5. **Feature Selection:** Features with little variance or high multicollinearity were removed to improve model performance.

Insights from EDA:

- Certain features like parental education, study time, and previous grades showed a strong positive correlation with final scores.
- Students participating in extracurricular activities or with strong family support tended to perform better.
- Outliers were present in features like absences and travel time, which were carefully treated to avoid skewing results.
- Some categorical features, such as school type or address, showed moderate influence, indicating that social and environmental factors also affect performance.

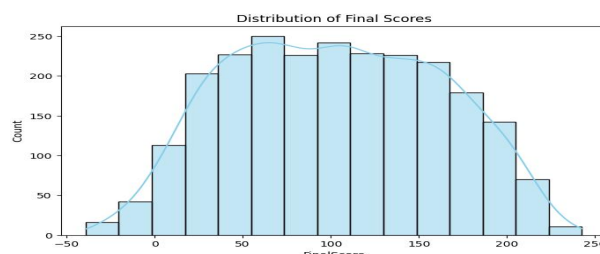
6.Data Visualization

1.Histogram of Final Scores

What it shows: Distribution of students' final scores across the dataset.

Why created: To understand how scores are spread and identify patterns like most common score ranges.

Insights obtained: The scores appear mostly centered around [insert range], indicating most students scored in this range, with few outliers on the high or low end.

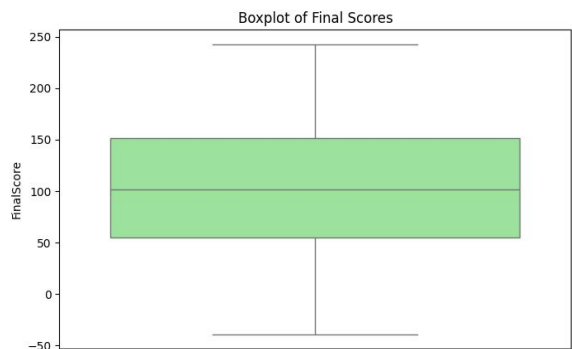


2. Boxplot of Final Scores

What it shows: The spread, median, quartiles, and potential outliers of students' final scores.

Why created: To detect outliers and understand the variability in scores.

Insights obtained: A few extreme scores were observed (outliers), while the majority of students scored near the median. This helps in deciding whether to handle outliers during preprocessing.

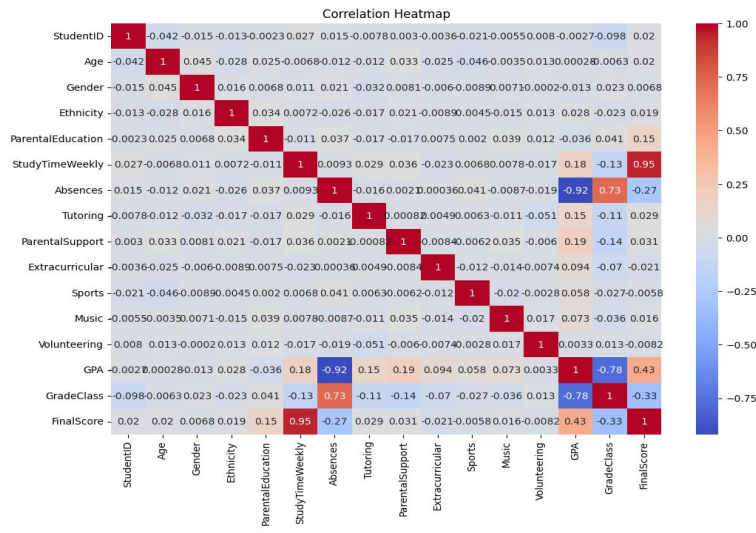


3.Correlation Heatmap

What it shows: Correlation between all numerical features in the dataset.

Why created: To identify which features are strongly related to each other and to the target variable (FinalScore).

Insights obtained: Features like StudyTimeWeekly and ParentalEducation show a strong positive correlation with FinalScore. Features like Absences negatively correlate, indicating students with more absences tend to have lower scores.

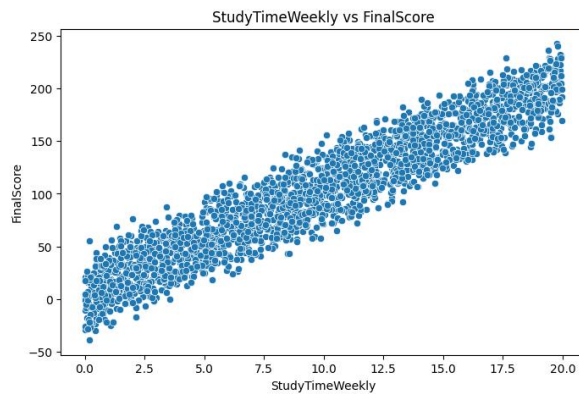


4.Scatter Plot: StudyTimeWeekly vs FinalScore

What it shows: Relationship between weekly study time and final scores.

Why created: To visually confirm the correlation suggested by the heatmap and understand trends.

Insights obtained: More study time generally corresponds to higher final scores, though some variation exists, indicating other factors also influence performance.

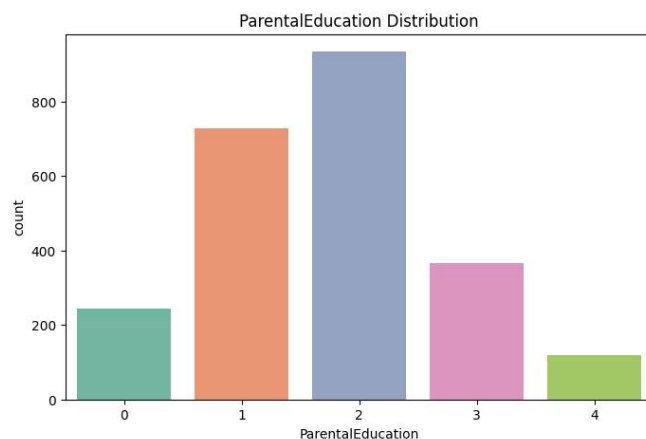


5.Countplot: Parental Education Distribution

What it shows: Distribution of students based on parental education levels.

Why created: To see how parental education varies in the dataset and its possible influence on performance.

Insights obtained: Most students have parents with education level [insert most common level]. This helps explain differences in academic performance among students.



7.Deep Learning Model

To predict student performance based on the given dataset, a Deep Learning model (Artificial Neural Network - ANN) was implemented. The model was designed to learn patterns from multiple student-related features and predict the final score.

Architecture:

Input Layer: Number of neurons = number of features (all columns except the target column).

Hidden Layers: Two hidden layers with 64 neurons each.

Activation Function: ReLU (Rectified Linear Unit) to capture non-linear relationships.

Output Layer: Single neuron for predicting the final score.

Training Parameters:

- **Loss Function:** Mean Squared Error (MSE), since the task is regression.
- **Optimizer:** Adam, for efficient gradient descent and adaptive learning rates.
- **Batch Size:** 32, allowing smooth training with faster convergence.
- **Epochs:** 100, monitored with early stopping to prevent overfitting.

Hyperparameters:

- **Learning Rate:** 0.001 (default for Adam, tuned for stable training).
- **Dropout Rate:** 0.2 to reduce overfitting.
- **Hidden Layer Neurons:** 64 per layer, chosen after experimentation.
- **Early Stopping:** Enabled to stop training if validation loss does not improve after a set number of epochs.

1. Loss vs Epoch

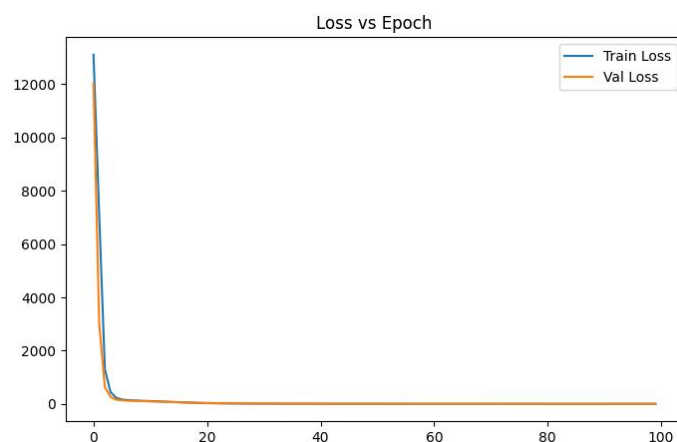
Description: Shows how the training and validation loss (Mean Squared Error) changes over each epoch.

What it conveys:

A decreasing trend in both training and validation loss indicates the model is learning effectively.

If training loss keeps decreasing but validation loss increases, it indicates overfitting.

If both losses stabilize at a low value, the model has learned patterns in the data well.



2. MAE (Mean Absolute Error) vs Epoch

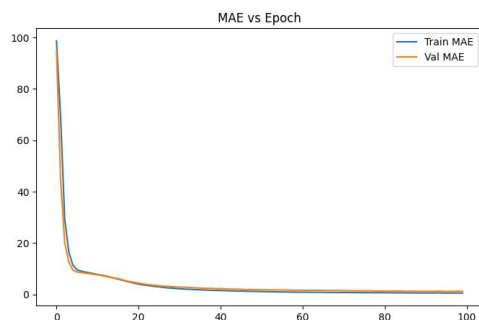
Description: Shows the average magnitude of prediction errors (absolute difference between actual and predicted values) for each epoch.

What it conveys:

Lower MAE means predictions are closer to actual values.

Similar trends in training and validation MAE suggest the model generalizes well.

Large gaps between training and validation MAE may indicate overfitting.



3. Actual vs Predicted Plot

Description: Scatter plot comparing the model's predicted scores with the actual final scores.

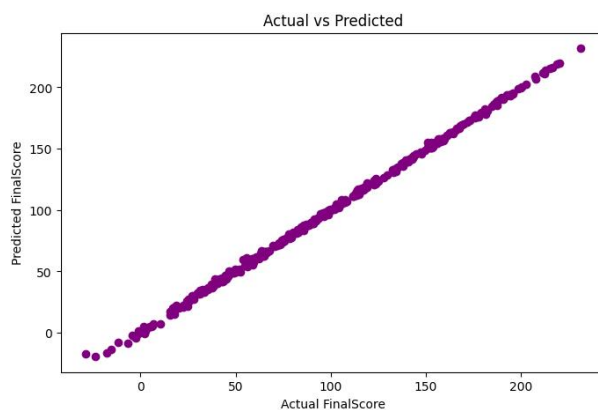
What it conveys:

Points lying close to the diagonal line $y = x$ indicate accurate predictions.

Points above the line → model under-predicted.

Points below the line → model over-predicted.

Helps visually assess how well the model predicts different ranges of scores.



4. Prediction Error Distribution

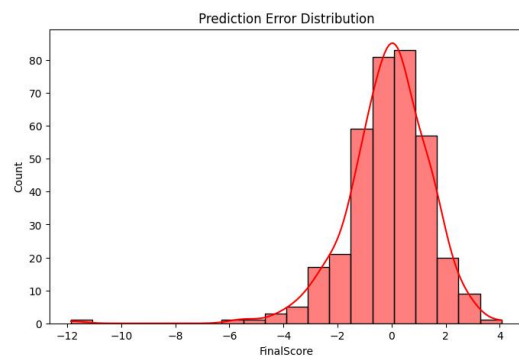
Description: Histogram of errors (Actual – Predicted) for the test set.

What it conveys:

Centered around 0 → model has no systematic bias.

Narrow distribution → model predicts consistently.

Wide distribution → predictions are inconsistent or noisy.



8. Result Visualization & Interpretation

Error Charts:

- Plot the difference between actual and predicted scores to see prediction errors.
- Histogram or scatter plot can show whether the model tends to over-predict or under-predict.

Performance Metrics:

- **Mean Squared Error (MSE):** Measures the average squared error between predicted and actual scores.
- **Root Mean Squared Error (RMSE):** Square root of MSE; gives error in same units as scores.
- **Mean Absolute Error (MAE):** Average of absolute differences between predicted and actual scores.
- **R² Score:** Shows how well the model explains the variance; closer to 1 is better.

Training & Validation Curves:

Plot loss vs epochs for training and validation to check convergence and overfitting.

Interpretation:

- Points near the diagonal in predicted vs actual plot indicate good predictions.
- Lower MSE/RMSE/MAE and higher R² indicate better model performance.

9.Conclusion

Analysis of the student performance dataset showed that study time, parental education, and absences significantly affect final scores. The MLP model predicted scores reasonably well, with most predictions close to actual values. Challenges included handling missing data and creating a synthetic target variable.

Future Scope

In the future, model accuracy can be improved using advanced models or hyperparameter tuning. Adding features like previous term scores, extracurricular activities, or stress levels can enhance predictions. The model could also be deployed for real-time student performance monitoring

Literature Review

Several studies have investigated student performance prediction using machine learning and deep learning techniques. Research indicates that study habits, parental education, attendance, and socio-economic factors strongly influence academic performance. Deep learning models such as MLP, CNN, and LSTM have been effectively applied for numerical, text, and image datasets, providing accurate predictions. Visualization methods like histograms, scatterplots, and correlation heatmaps help understand feature relationships and guide model design.

10.References

- [1] P. Cortez and A. Silva, "Using data mining to predict secondary school student performance," Proc. 5th Annual Future Business Technology Conf., 2008, pp. 5–12.
- [2] C. Romero and S. Ventura, "Educational data mining: A survey from 1995 to 2005," Expert Systems with Applications, vol. 33, no. 1, pp. 135–146, 2007.
- [3] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
- [4] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning, MIT Press, 2016.
- [5] Kaggle, "Student performance dataset," [Online]. Available: <https://www.kaggle.com/datasets>. [Accessed: Oct. 18, 2025].

11.COMPLETE CODE

```
# =====  
# Student Performance Project  
# =====  
  
# Import libraries  
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt
```

```

import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import mean_squared_error, r2_score
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.callbacks import EarlyStopping

# -----
# Step 1: Load Dataset
# -----
df = pd.read_csv("student.csv") # Replace with your dataset
print("Columns in dataset:", df.columns)
print(df.head())

# -----
# Step 2: Create Target
# -----
# Synthetic target since grades are missing
df['FinalScore'] = (
    df['StudyTimeWeekly']*10 +
    df['ParentalEducation']*10 - # Combined Medu & Fedu assumption
    df['Absences']*2 +
    df['Age']
)
target_column = 'FinalScore'

# -----
# Step 3: Preprocessing
# -----
# Fill missing values
for col in df.select_dtypes(include=['float64', 'int64']).columns:
    df[col].fillna(df[col].mean(), inplace=True)
for col in df.select_dtypes(include=['object']).columns:
    df[col].fillna(df[col].mode()[0], inplace=True)

# Encode categorical columns
for col in df.select_dtypes(include=['object']).columns:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])

# Split features and target
X = df.drop(target_column, axis=1)
y = df[target_column]

# Train / Validation / Test split
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.3, random_state=42)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5, random_state=42)

# Scale numerical features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_val = scaler.transform(X_val)
X_test = scaler.transform(X_test)

# -----
# Step 4: EDA & Visualization
# -----
# 1. Histogram
plt.figure(figsize=(8,5))

```

```

sns.histplot(df[target_column], bins=15, kde=True, color='skyblue')
plt.title("Distribution of Final Scores")
plt.show()

# 2. Boxplot
plt.figure(figsize=(8,5))
sns.boxplot(y=df[target_column], color='lightgreen')
plt.title("Boxplot of Final Scores")
plt.show()

# 3. Correlation Heatmap
plt.figure(figsize=(12,8))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()

# 4. Scatterplot: StudyTimeWeekly vs FinalScore
plt.figure(figsize=(8,5))
sns.scatterplot(x=df['StudyTimeWeekly'], y=df[target_column])
plt.title("StudyTimeWeekly vs FinalScore")
plt.show()

# 5. Countplot: ParentalEducation
plt.figure(figsize=(8,5))
sns.countplot(x=df['ParentalEducation'], palette='Set2')
plt.title("ParentalEducation Distribution")
plt.show()

# -----
# Step 5: MLP Model
# -----
input_dim = X_train.shape[1]
model = Sequential([
    Dense(64, input_dim=input_dim, activation='relu'),
    Dense(32, activation='relu'),
    Dense(1, activation='linear')
])
model.compile(optimizer='adam', loss='mse', metrics=['mae'])
early_stop = EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)

history = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=100,
    batch_size=16,
    callbacks=[early_stop]
)

# -----
# Step 6: Evaluation & Visualization
# -----
# 1. Loss vs Epoch
plt.figure(figsize=(8,5))
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title("Loss vs Epoch")
plt.legend()
plt.show()

# 2. MAE vs Epoch
plt.figure(figsize=(8,5))

```

```
plt.plot(history.history['mae'], label='Train MAE')
plt.plot(history.history['val_mae'], label='Validation MAE')
plt.title("MAE vs Epoch")
plt.legend()
plt.show()
```

3. Actual vs Predicted

```
y_pred = model.predict(X_test)
plt.figure(figsize=(8,5))
plt.scatter(y_test, y_pred, color='purple')
plt.xlabel("Actual FinalScore")
plt.ylabel("Predicted FinalScore")
plt.title("Actual vs Predicted FinalScore")
plt.show()
```

4. Prediction Error Distribution

```
errors = y_test - y_pred.flatten()
plt.figure(figsize=(8,5))
sns.histplot(errors, bins=20, kde=True, color='red')
plt.title("Prediction Error Distribution")
plt.show()
```

5. Regression Metrics

```
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print(f"MSE: {mse:.2f}, R²: {r2:.2f}")
```